

Lab 1

24341272.1 — the Lexer (Flex)

Header section (lines 1–28)

Line(s)	Explanation
<code>%option noyywrap</code>	Tells Flex not to require a <code>yywrap()</code> function — the scanner stops at EOF instead of asking to open another file.
<code>%{ ... %}</code> (3–28)	Raw C/C++ code copied verbatim into the generated <code>lex.yy.c</code> .
<code>#include <bits/stdc++.h></code>	Pulls in all standard C++ library headers (competitive-programming style shortcut).
<code>#include "symbol_info.h"</code>	Brings in the <code>symbol_info</code> class used to carry token name/type between lexer and parser.
<code>#define YYSTYPE</code> <code>symbol_info*</code>	Redefines Bison's semantic value type (<code>YYSTYPE</code>) from the default <code>int</code> to a pointer to <code>symbol_info</code> , so every token can carry a lexeme + type.
<code>#include "y.tab.h"</code>	Includes the token definitions (<code>IF</code> , <code>ID</code> , <code>RPAREN</code> , etc.) generated by Bison from the <code>.y</code> file — this is why <code>.l</code> depends on <code>.y</code> being compiled first.
<code>extern YYSTYPE yylval;</code>	Declares the global variable Bison uses to receive a token's semantic



<code>extern YYSTYPE yylval;</code>	Declares the global variable Bison uses to receive a token's semantic value; defined in the parser file.
<code>void yyerror(char *);</code>	Forward declaration of the error-reporting function (defined in the <code>.y</code> file).
<code>string loglist;</code>	A string buffer used to build each log message before writing it out.
<code>extern int line_num;</code>	Line counter, declared here but defined/initialized in the <code>.y</code> file (shared global).
<code>extern ofstream outlog;</code>	The output log file stream, also owned by the <code>.y</code> file.

Regular definitions (lines 29–43)

These are named patterns reused later via `{name}`:

- `delim [ \t\v\r ]` — a single whitespace-ish character (space, tab, vertical tab, carriage return).
- `ws {delim}+` — one or more whitespace chars (to be skipped).
- `newline \n` — literal newline (used to bump `line_num`).
- `letter_ [A-Za-z_]` — a letter or underscore (identifier start).
- `digit [0-9]` — a single digit.
- `id {letter_}({letter_}|{digit})*` — identifier: starts with a letter/underscore, followed by any mix of letters/digits/underscores.
- `int_lit {digit}+` — integer literal: one or more digits.
- `exp [Ee][+-]?{digit}+` — an exponent part like `e+10` or `E-3`, used inside float literals.
- `float_lit (...)` — three alternative float shapes: `digits.digits[exp]?`, `.digits[exp]?`, or `digits+exp` (i.e., an integer with a mandatory exponent, like `2e5`).

- `"++" / "--"` → increment/decrement operators, tagged `INCOP / DECOP`.
- `"<=" | ">=" | "==" | "!=" | "<" | ">"` → all relational operators collapse into one token type `RELOP`.
- `"&&" | "||"` → logical operators → `LOGICOP`.
- `"+" | "-"` → additive operators → `ADDOP`.
- `"*" | "/" | "%"` → multiplicative operators → `MULOP`.
- `"="` → assignment → `ASSIGNOP` (no `symbol_info` created — parser doesn't need the lexeme).
- `"!"` → logical NOT → `NOT`.
- `"(" "}" "{" "}" "[" "]" " , " : " ; "` → each punctuation/bracket character maps to its own token (`LPAREN`, `RPAREN`, `LCURL`, `RCURL`, `LTHIRD`, `RTHIRD`, `COMMA`, `COLON`, `SEMICOLON`) — just logged, nothing else, since the parser only needs to know *which* punctuation it was, not the character itself.
- `.` (catch-all) → matches any other single character; logs it as "Unrecognized character" and does not return a token (so it's effectively skipped/ignored by the parser, only recorded for diagnostics).

24341272.y — the Parser (Bison/Yacc)

Header section (lines 1-27)

- Same `bits/stdc++.h`, `symbol_info.h`, and `YYSTYPE` redefinition as the lexer, so both files agree on the semantic value type.
- `int yyparse(void); / int yylex(void);` — forward declarations of the parser entry point and the lexer function it will call.
- `extern FILE *yyin;` — the input file pointer that Flex reads from; set in `main`.
- `ofstream outlog;` — this is where the log stream is actually defined (the `.l` file just declares it `extern`).
- `int line_num = 1;` — this is where the line counter is actually defined, initialized to 1 (the `.l` file only declares it `extern`).
- `void yyerror(const char *s);` — forward declaration; implemented at the bottom.

Token declarations (lines 29-33)

- `%token IF ELSE FOR ... PRINTLN` — keyword tokens.
- `%token INT FLOAT VOID CHAR DOUBLE` — type-name tokens.
- `%token ID CONST_INT CONST_FLOAT` — tokens that carry a lexeme.
- `%token ADDOP MULOP INCOP DECOP RELOP ASSIGNOP LOGICOP NOT` — operator tokens.
- `%token LPAREN RPAREN LCURL RCURL LTHIRD RTHIRD COMMA COLON SEMICOLON` — punctuation tokens.

Precedence/associativity (lines 35-36)

```
%nonassoc LOWER_THAN_ELSE
%nonassoc ELSE
```

This is the standard **dangling-else fix**: `LOWER_THAN_ELSE` is a dummy, lower-precedence terminal. The `if (...) statement` rule is tagged `%prec LOWER_THAN_ELSE` (see line 276) so Bison prefers to *shift* an upcoming `ELSE` (attaching it to the nearest `if`) rather than *reduce* the plain `if`-rule early — resolving the if/else ambiguity in favor of matching each `else` to the closest unmatched `if`.

Grammar rules (lines 38-566)

Every rule follows the same pattern: match a right-hand side, log which rule fired plus the reconstructed source text, then build a new `symbol_info` combining the children's text (essentially re-serializing the parsed code while validating its grammar) and assign it to `$$`. Key productions, top-down:

- `start : program` — grammar's start symbol; logs completion.
- `program : program unit | unit` — a program is one or more top-level "units," built by left-recursive concatenation (newline-joined text of each unit).
- `unit : variable_decl | func_definition` — a top-level unit is either a global variable declaration or a function definition.
- `func_definition` — `type ID (params) { ... }` or `type ID () { ... }`, reconstructs as `"type name(params)\nbody"`.
- `param_list` — recursively builds a comma-separated parameter list (`type_specifier ID`, or bare `type_specifier` with no name, i.e. K&R-style unnamed params).
- `compound_statement` — `{ statements }` or empty `{ }`, wraps text in braces.
- `variable_decl : type_specifier declaration_list SEMICOLON` — e.g. `int a, b;`.
- `type_specifier : INT|FLOAT|VOID|CHAR` — maps token to the literal type-name string.
- `declaration_list` — comma-separated identifiers, with optional array-size suffix `ID[CONST_INT]`.

- `statements : statement | statements statement` — a block body is one or more statements (newline-joined).
- `statement` — the big alternative list:
 - `variable_decl`, `expression_statement`, `compound_statement` (nested blocks)
 - `FOR (expr_stmt expr_stmt expr) statement` — classic C for-loop
 - `IF (expr) statement %prec LOWER_THAN_ELSE` — if without else (dangling-else resolved as above)
 - `IF (expr) statement ELSE statement` — if/else
 - `WHILE (expr) statement`
 - `PRINTLN LPAREN ID RPAREN SEMICOLON` — `printf(x);`-style call limited to a single identifier
 - `RETURN expression SEMICOLON`

- `expression_statement : SEMICOLON | expression SEMICOLON` — an expression followed by `;`, or an empty statement (`;`).
- `variable : ID | ID [ expression ]` — a plain variable or an array access.
- `expression : logic_expression | variable ASSIGNOP logic_expression` — assignment binds an lvalue (`variable`) to a logic expression.
- `logic_expression : rel_expression | rel_expression LOGICOP rel_expression` — handles `&&/||` (non-chaining; only one operator per rule, so `a&&b&&c` isn't directly expressible — a known simplification typical of these course grammars).
- `rel_expression : simple_expression | simple_expression RELOP simple_expression` — handles `<, >, <=, >=, ==, !=`, again single-level (no chaining).
- `simple_expression : term | simple_expression ADDOP term` — left-recursive, handles `+/-` with proper left-associativity.
- `term : unary_expression | term MULOP unary_expression` — left-recursive, handles `*, /, %`.
- `unary_expression : ADDOP unary_expression | NOT unary_expression | factor_info` — unary plus/minus and logical negation.
- `factor_info : factor` — a thin pass-through wrapper around `factor` (adds a grammar layer, likely for symbol-table bookkeeping elsewhere in the original assignment).
- `factor` — the base case of the expression grammar:
 - `variable`
 - `ID LPAREN argument_list RPAREN` — function call
 - `LPAREN expression RPAREN` — parenthesized expression
 - `CONST_INT / CONST_FLOAT` — literals
 - `variable INCOP / variable DECOP` — postfix `++/--`

- `factor` — the base case of the expression grammar:
 - `variable`
 - `ID LPAREN argument_list RPAREN` — function call
 - `LPAREN expression RPAREN` — parenthesized expression
 - `CONST_INT / CONST_FLOAT` — literals
 - `variable INCOP / variable DECOP` — postfix `++/--`
- `argument_list : arguments | ε` — call arguments, or none.
- `arguments : arguments COMMA logic_expression | logic_expression` — comma-separated argument list, left-recursive.

Epilogue (lines 568–605)

```
c
int main(int c, char *v[])
```

- Requires exactly one command-line argument (the source file); otherwise prints usage and exits.
- Opens that file as `yyin` (Flex's global input stream).
- Opens `24341272_log.txt` in truncate mode as the log output.
- If the input file failed to open, reports an error and exits.
- Calls `yyparse()`, which drives the whole lex/parse process (`yylex()` is called internally each time the grammar needs the next token).
- After parsing, writes the total line count to the log and closes both the log file and input file.

```
c
void yyerror(const char *s)
```

- Bison calls this automatically on a syntax error, passing a default message (`s`, e.g. `"syntax error"`). Here it just logs the current line number and message to `outlog` rather than aborting or printing to `stderr`.

How the two files interact: `.l` tokenizes the raw source character-by-character/regex-by-regex, packaging identifiers/literals into `symbol_info` objects passed via `yyval`; `.y` consumes that token stream, applies C-grammar rules, and — as a side effect of each reduction — reconstructs/logs a clean textual representation of the parsed program while validating that its structure is syntactically legal. `y.tab.h` (generated from `.y`) is the glue that lets `.l` know the numeric codes for `IF`, `ID`, `RPAREN`, etc.

Lab 2

3.1 `symbol_info.h`

```
cpp
#include <bits/stdc++.h>
using namespace std;
```

Pulls in the whole standard library (a common competitive-programming shortcut; not ideal style for production code, but fine for a lab).

```
cpp
class symbol_info
{
private:
    string name;
    string type;
    string symbol_kind;
    string data_type;
    int array_size;
    vector<pair<string, string>> parameters;
```

Fields:

- `name` — the identifier's text (e.g. `"a"`).
- `type` — the *token* type used during parsing (e.g. `"ID"`, `"ADDOP"`) — this is really a lexer/parser bookkeeping field, distinct from `data_type`.
- `symbol_kind` — `"variable"`, `"array"`, or `"function"` — set once you know what role the symbol plays.
- `data_type` — the C data type (`"int"`, `"float"`, etc.).
- `array_size` — size if it's an array, `-1` sentinel otherwise (means "not an array").
- `parameters` — for functions: a list of `(type, name)` pairs, one per parameter.

Constructor: every `symbol_info` is created knowing at least its name and (lexer) type; the rest default to empty/unset until the grammar action code fills them in via setters.

cpp

```
string get_name() { return name; }
string get_type() { return type; }
void set_name(string name) { this->name = name; }
void set_type(string type) { this->type = type; }
```

Standard getters/setters for `name` / `type`.

cpp

```
void set_symbol_kind(string symbol_kind) { this->symbol_kind = symbol_kind; }
string get_symbol_kind() { return symbol_kind; }
void set_data_type(string data_type) { this->data_type = data_type; }
string get_data_type() { return data_type; }
void set_array_size(int array_size) { this->array_size = array_size; }
int get_array_size() { return array_size; }
```

Same pattern for `symbol_kind`, `data_type`, `array_size`.

cpp

```
void add_parameter(string parameter_type, string parameter_name)
{
    parameters.push_back({parameter_type, parameter_name});
}
void set_parameters(const vector<pair<string, string>> &params)
{
    parameters = params;
}
vector<pair<string, string>> get_parameters() { return parameters; }
```


`add_parameter` appends one `(type, name)` pair one at a time; `set_parameters` replaces the whole vector at once (this is the one actually used in the `.y` file — it copies the globally-accumulated `paramCollection` in one shot when a function symbol is created).

cpp

```
~symbol_info() {}
```

Empty destructor — `symbol_info` doesn't own any heap memory itself (the vector/strings clean themselves up), so nothing extra to release. (Note: this class doesn't have a copy constructor/assignment operator defined, but since `scope_table` / `symbol_table` only ever store `symbol_info*` pointers rather than copies of the object, that's not an issue here.)

3.2 scope_table.h

```
cpp
#include "symbol_info.h"

class scope_table
{
private:
    int bucket_count;
    int unique_id;
    scope_table *parent_scope = NULL;
    vector<list<symbol_info *>> table;
```

- `bucket_count` — number of hash buckets.
- `unique_id` — this scope's ID (printed in the log, e.g. "ScopeTable # 2").
- `parent_scope` — pointer to the enclosing scope table (NULL for the outermost/global scope) — this is what turns the list of scope tables into a chain you can walk for lookups.
- `table` — the actual hash table: a vector of buckets, each bucket a `list` of pointers (chaining to resolve collisions).

```
cpp

int hash_function(string name)
{
    int sum = 0;
    for (char ch : name) { sum += ch; }
    return sum % bucket_count;
}
```

A simple additive hash: sums the ASCII values of every character in the name, then reduces mod `bucket_count` to land in a valid bucket index. Cheap but collision-prone for anagrams (e.g. `"ab"` and `"ba"` hash identically) — acceptable for a lab exercise, though a polynomial hash (e.g. multiply-and-add with a prime) would spread things better.

```
cpp
public:
    scope_table();
    scope_table(int bucket_count, int unique_id, scope_table *parent_scope);
    scope_table *get_parent_scope();
    int get_unique_id();
    symbol_info *lookup_in_scope(symbol_info* symbol);
    bool insert_in_scope(symbol_info* symbol);
    bool delete_from_scope(symbol_info* symbol);
    void print_scope_table(ofstream& outlog);
    ~scope_table();
};
```

Declares the public interface matching exactly what the Lab 2 spec asked for: Insert, Look up, Delete, Print, plus a parent-scope accessor and constructors/destructor.

Declares the public interface matching exactly what the Lab 2 spec asked for: Insert, Look up, Delete, Print, plus a parent-scope accessor and constructors/destructor.

```
cpp
scope_table::scope_table()
{
    bucket_count = 0;
    unique_id = 0;
    parent_scope = NULL;
}
```

Default constructor — leaves the table with zero buckets (not really used in practice; the parameterized constructor below is what's actually called).

```
cpp
scope_table::scope_table(int bucket_count, int unique_id, scope_table *parent)
{
    this->bucket_count = bucket_count;
    this->unique_id = unique_id;
    this->parent_scope = parent_scope;
    table.resize(bucket_count);
}
```

The real constructor: stores the three parameters and allocates `bucket_count` empty buckets (`table.resize` creates that many empty `list<symbol_info*>` slots).

The real constructor: stores the three parameters and allocates `bucket_count` empty buckets (`table.resize` creates that many empty `list<symbol_info*>` slots).

cpp

```
scope_table *scope_table::get_parent_scope() { return parent_scope; }
int scope_table::get_unique_id() { return unique_id; }
```

Simple accessors.

cpp

```
symbol_info *scope_table::lookup_in_scope(symbol_info* symbol)
{
    if (!symbol) return NULL;
    int bucket_idx = hash_function(symbol->get_name());
    for (symbol_info* sym : table[bucket_idx])
    {
        if (sym->get_name() == symbol->get_name()) { return sym; }
    }
    return NULL;
}
```

Hash the given symbol's name to find the right bucket, then linearly scan that bucket's chain comparing names; return the matching stored `symbol_info*` or `NULL` if not found *in this scope specifically* (no parent-walking here — that's `symbol_table::lookup`'s job).

Hash the given symbol's name to find the right bucket, then linearly scan that bucket's chain comparing names; return the matching stored `symbol_info*` or `NULL` if not found *in this scope specifically* (no parent-walking here — that's `symbol_table::lookup`'s job).

```
cpp

bool scope_table::insert_in_scope(symbol_info* symbol)
{
    if (!symbol) return false;
    symbol_info tmp(symbol->get_name(), symbol->get_type());
    if (lookup_in_scope(&tmp) != NULL) { return false; }
    int bucket_idx = hash_function(symbol->get_name());
    table[bucket_idx].push_back(symbol);
    return true;
}
```

Duplicate check first: builds a throwaway `tmp` `symbol_info` with the same name and re-uses `lookup_in_scope` to see if it's already present in *this* scope table (redeclaration check); if so, reject the insert (`false`). Otherwise hash and append to the correct bucket, return `true`. Note this stores the raw pointer `symbol` — the scope table takes ownership (its destructor deletes it).

Duplicate check first: builds a throwaway `tmp` `symbol_info` with the same name and re-uses `lookup_in_scope` to see if it's already present in *this* scope table (redeclaration check); if so, reject the insert (`false`). Otherwise hash and append to the correct bucket, return `true` . Note this stores the raw pointer `symbol` — the scope table takes ownership (its destructor deletes it).

```
cpp
bool scope_table::delete_from_scope(symbol_info* symbol)
{
    if (!symbol) return false;
    int bucket_idx = hash_function(symbol->get_name());
    for (auto iter = table[bucket_idx].begin(); iter != table[bucket_idx].end(); ++iter)
    {
        if ((*iter)->get_name() == symbol->get_name())
        {
            delete *iter;
            table[bucket_idx].erase(iter);
            return true;
        }
    }
    return false;
}
```

Finds the bucket, iterates looking for a name match, and if found `delete`s the pointed-to `symbol_info` (freeing memory) and erases the list entry, returning `true` ; returns `false` if never found.

3.3 symbol_table.h

```
cpp

#include "scope_table.h"

class symbol_table
{
private:
    scope_table *current_scope;
    int bucket_count;
    int current_scope_id;
public:
    symbol_table(int bucket_count);
    ~symbol_table();
    void enter_scope();
    void exit_scope();
    bool insert(symbol_info* symbol);
    bool remove(symbol_info* symbol);
    symbol_info* lookup(symbol_info* symbol);
    void print_current_scope(ofstream& outlog);
    void print_all_scopes(ofstream& outlog);
    int get_current_scope_id();
};
```

`current_scope` is the pointer to the topmost scope table (the "top of stack");
`bucket_count` is remembered so every new scope table is created with the same bucket size; `current_scope_id` is a running counter for assigning unique IDs.

```
cpp
symbol_table::symbol_table(int bucket_count)
{
    this->bucket_count = bucket_count;
    this->current_scope_id = 1;
    current_scope = new scope_table(bucket_count, current_scope_id, NULL);
}
```

Constructor creates the initial **global scope** (ID 1, no parent) — matching the spec's example where global scope is numbered 1.

```
cpp
symbol_table::~symbol_table()
{
    while (current_scope != NULL)
    {
        scope_table *temp = current_scope;
        current_scope = current_scope->get_parent_scope();
        delete temp;
    }
}
```

Destructor walks the whole chain from current scope up through every parent, deleting each `scope_table` (which in turn deletes all its symbols) — cleans up everything at program end.

Symbol_info.h

`using namespace std;`

This directive tells the compiler to bring all the names from the standard C++ namespace (`std`) into the current global scope.

`parameters` — for functions, a list of (type, name) pairs describing its parameter list

The `this->` pointer is used to refer to the object's internal member variables. Because the constructor's parameters have the exact same names as the class members (`name` and `type`),

`~symbol_info() {}`; ———> empty destructor

Scope_table.h

Each `scope_table` represents one nested block/scope (global scope, a function body, an `if/for` block, etc.):

The `vector` represents the array of buckets.

Constructor —> default constructor + Real constructor

`.resize(bucket_count)` forces the vector to allocate exactly that many empty linked lists in memory immediately. \

1) `lookup_in_scope`, is responsible for finding a specific variable or function within a single scope (like inside a specific function block or `if` statement).

`scope_table` uses a hash table to store symbols, this function doesn't need to search through every single variable in the table. Instead, it uses the hash to jump straight to where the symbol should be.

`symbol_info *scope_table::lookup_in_scope(symbol_info* symbol)` This defines the method. It takes a pointer to a `symbol_info` object (which contains the name we are looking for) and promises to return a pointer to the matching `symbol_info` object if it finds one in the table.

`if (!symbol) return NULL;` This is a safety check. If the pointer passed into the function is null (empty), the function immediately stops and returns `NULL` to prevent the program from crashing.

`int bucket_idx = hash_function(symbol->get_name());` This is where the hash table's efficiency kicks in. It gets the name of the symbol we are searching for (e.g., "x") and passes it through the `hash_function`. The hash function calculates and returns a specific integer index (`bucket_idx`). If "x" exists in this scope, it is guaranteed to be inside this specific bucket.

`for (symbol_info* sym : table[bucket_idx])` Because multiple variables might hash to the exact same bucket index (a collision), each bucket is actually a linked list. This `for` loop iterates through every symbol currently stored in that specific list.

if (sym->get_name() == symbol->get_name()) As it iterates through the list, it compares the name of the currently stored symbol (**sym**) with the name of the symbol we are searching for.

return sym; If the names match exactly, the search is successful! It immediately returns the pointer to the stored symbol (**sym**) so the compiler can access its data (like its data type or array size).

return NULL; If the **for** loop finishes checking every item in that bucket and hasn't found a match, it means the symbol was never declared in this specific scope. It returns **NULL** to indicate a failed search.

This method, **insert_in_scope**, is responsible for adding a newly declared variable or function into the current scope's hash table. A critical part of its job is ensuring that a program does not try to declare two things with the exact same name in the same block (for example, **int x;** and **float x;** in the same function).

Here is the line-by-line breakdown of how it works:

- **bool scope_table::insert_in_scope(symbol_info* symbol)** This defines the method. It takes a pointer to the **symbol_info** object you want to add to the table. It returns a boolean (**true** or **false**) to tell the caller whether the insertion was successful or if it failed.
- **if (!symbol) return false;** This is a standard safety check. If a null (empty) pointer is passed to the function by mistake, it immediately aborts and returns **false** to prevent the program from crashing.
- **symbol_info tmp(symbol->get_name(), symbol->get_type());** This creates a temporary **symbol_info** object named **tmp**. It extracts the name and type from the provided **symbol** pointer and uses them to initialize this new temporary object.
- **if (lookup_in_scope(&tmp) != NULL)** Before inserting the new symbol, the compiler must ensure it doesn't already exist. It passes the memory address of the temporary object (**&tmp**) to the **lookup_in_scope** method we looked at previously.
- **return false; // Already exists** If **lookup_in_scope** returns anything other than **NULL**, it means a symbol with this exact name has already been declared in this specific block. The function aborts the insertion and returns **false** to indicate a duplicate declaration error.
- **int bucket_idx = hash_function(symbol->get_name());** If the code reaches this line, it means the symbol is unique in this scope. The function takes the symbol's name and runs it through the **hash_function** to figure out exactly which bucket index it belongs in.
- **table[bucket_idx].push_back(symbol);** It accesses the linked list located at the calculated **bucket_idx** inside the hash table and uses **push_back** to append the new symbol pointer to the very end of that list.

- **return true;** The symbol has been successfully stored in the hash table. The function returns **true** so the compiler knows the insertion worked.

This method, `delete_from_scope`, is responsible for finding a specific symbol in the current scope's hash table and safely removing it from memory.

Here is the line-by-line breakdown of how it works:

- **bool scope_table::delete_from_scope(symbol_info* symbol)** This defines the method. It takes a pointer to the `symbol_info` object that you want to remove and returns a boolean (**true** or **false**) to indicate whether the deletion was successful.
- **if (!symbol) return false;** This is a safety check. If an empty (null) pointer is passed into the function, there is nothing to delete, so it immediately aborts and returns **false**.
- **int bucket_idx = hash_function(symbol->get_name());** Just like in the lookup and insert methods, the function takes the symbol's name and runs it through the `hash_function`. This tells the program exactly which bucket in the hash table to check, avoiding the need to search the entire table.
- **for (auto iter = table[bucket_idx].begin(); iter != table[bucket_idx].end(); ++iter)** This sets up a loop to iterate through the linked list located at that specific `bucket_idx`. It uses an iterator (`iter`), which acts like a specialized pointer that moves from one item in the list to the next.
- **if ((*iter)->get_name() == symbol->get_name())** Inside the loop, it dereferences the iterator (`*iter`) to look at the current stored symbol, and checks if its name matches the name of the symbol we want to delete.
- **delete *iter;** If a match is found, this line actually deletes the `symbol_info` object from the computer's memory. Because these symbols were created dynamically (using the `new` keyword elsewhere in the code), you must manually **delete** them here to prevent memory leaks.
- **table[bucket_idx].erase(iter);** Even though the memory was freed in the previous line, the linked list still holds a "dangling pointer" to that now-empty memory space. The `.erase(iter)` command safely removes that node entirely from the list.
- **return true;** The symbol has been successfully deleted and removed from the table. The function returns **true** to let the caller know it worked.
- **return false;** If the loop finishes checking every item in the bucket and never found a matching name, it means the symbol wasn't in the table to begin with. It returns **false** to indicate the deletion failed.

cpp



```
scope_table::~scope_table()
{
    for (int i = 0; i < bucket_count; i++)
    {
        for (symbol_info *symbol : table[i])
        {
            delete symbol;
        }
        table[i].clear();
    }
    table.clear();
}
```

Destructor: iterates every bucket, deletes every `symbol_info*` it owns (avoiding memory leaks), clears each bucket's list, then clears the `table` vector.

Here is the line-by-line breakdown of how it accomplishes this:

- **`void scope_table::print_scope_table(ofstream& outlog)`** Defines the method, passing an output file stream (`ofstream`) reference named `outlog` so the function can write directly to the log file instead of the console.
H
- **`outlog << "ScopeTable # " << unique_id << endl;`** Prints the header for the table, showing its dynamically assigned ID (e.g., "ScopeTable # 2") to help track which scope block is currently being logged.
H
- **`for (int i = 0; i < bucket_count; i++)`** Starts a loop that will iterate through every single bucket index in the hash table, from `0` up to one less than `bucket_count`.
H
- **`if (table[i].empty()) { continue; }`** Checks if the linked list at the current bucket index `i` is empty. If it is, `continue` skips the rest of the loop and moves to the next bucket, preventing the log file from being cluttered with empty bucket indices.
H+ 1
- **`outlog << i << " --> " << endl;`** If the bucket is not empty, this prints the bucket's index number followed by an arrow (e.g., `5 -->`), signaling that the following lines will list the symbols stored at this hash index.
H

- **for (symbol_info *symbol : table[i])** Starts an inner loop that iterates through every `symbol_info` pointer stored within the linked list of the current bucket.
H
- **outlog << "< " << symbol->get_name() << " : " << symbol->get_type() << " >" << endl;** Prints the fundamental token information in an angle-bracket format. For example, if the symbol is a variable named `x`, it might print `< x : ID >`.
H+ 1
- **if (symbol->get_symbol_kind() == "variable")** Checks if the parser tagged this symbol specifically as a standard variable.
H
- **outlog << "Variable" << endl;** Prints the label "Variable" to categorize it.
H
- **outlog << "Type: " << symbol->get_data_type() << endl;** Retrieves and prints the semantic data type (like `int` or `float`) that the variable holds.
H+ 1
- **else if (symbol->get_symbol_kind() == "array")** If it wasn't a standard variable, it checks if the symbol was tagged as an array.
H
- **outlog << "Array" << endl;** Prints the label "Array".
H
- **outlog << "Type: " << symbol->get_data_type() << endl;** Prints the underlying data type of the array's elements.
H
- **outlog << "Size: " << symbol->get_array_size() << endl;** Retrieves and prints the integer size of the array (e.g., the `10` in `int arr[10];`).
H+ 1
- **else if (symbol->get_symbol_kind() == "function")** If it wasn't a variable or array, it checks if the symbol represents a function definition.
H
- **vector<pair<string, string>> parameters = symbol->get_parameters();** Calls the symbol's getter to retrieve its parameter list. This list is returned as a vector of string pairs (where the first string is the type, and the second is the variable name).
H+ 3
- **outlog << "Function Definition" << endl;** Prints the label "Function Definition".
H
- **outlog << "Return Type: " << symbol->get_data_type() << endl;** Prints the function's return type (e.g., `void`, `int`).
H

- `outlog << "Number of Parameters: " << parameters.size() << endl;`
Calculates how many parameters the function takes by checking the `.size()` of the retrieved vector, and prints that number.
H
- `outlog << "Parameter Details: ";` Starts the line that will list the individual parameters.
H
- `for (int j = 0; j < parameters.size(); j++)` Starts a standard integer-based loop to iterate through the parameter vector.
H
- `outlog << parameters[j].first;` Prints the first element of the pair, which is the parameter's type (e.g., `int`).
H
- `if (parameters[j].second != "") { outlog << " " << parameters[j].second; }` Checks if the parameter actually has a name. (In C++, a function can be declared with just types, like `void func(int, float)`). If a name exists, it prints it after a space.
H+ 1
- `if (j < parameters.size() - 1) { outlog << ", "; }` Checks if the current parameter is *not* the last one in the list. If there are more parameters to come, it adds a comma and a space to separate them.
H+ 1
- `outlog << endl;` Prints a newline to end the parameter details line.
H
- `outlog << endl;` Prints one final empty newline to create visual separation between this symbol's block of information and the next symbol in the bucket.
H

[Script.sh](#)

Runs `yacc` on the grammar file. `-d` generates a header (`y.tab.h`) with token definitions; `-y` forces POSIX-standard output names (`y.tab.c/y.tab.h`); `--debug` `--verbose` produce extra trace info and a `.output` file describing parser states (useful for debugging shift/reduce conflicts).

.y or Bison File

```
51          if(symtbl) symtbl->print_all_scopes(outlog);
```

L51: if the symbol table manager exists, prints every remaining scope (at this point, only the global scope, since all function-local scopes were already popped and printed when their `compound_statement` closed).

Lab 3

This document explains **every section** of the Yacc/Bison specification, in the order it appears in the file: the C++ preamble, token declarations, and every grammar rule with its semantic actions.

1. The Preamble (`%{ ... %}`)

```
c
#include "symbol_table.h"
#define YYSTYPE symbol_info*
```

- Pulls in the symbol-table classes built in the previous assignment (`symbol_info` , `scope_table` , `symbol_table`).
- Redefines Bison's value-stack type `YYSTYPE` to be a pointer to `symbol_info`. This is the backbone of the whole semantic analyzer: every terminal and non-terminal on the parse stack carries a full `symbol_info*` (name string, node-kind, data type, symbol kind, parameter list, array size, etc.), not just a token string. This lets type information flow up the parse tree naturally through `$1` , `$2` , `$$` .

```
c
extern FILE *yyin;
int yyparse(void);
int yylex(void);
extern YYSTYPE yylval;
```

Standard forward declarations connecting to the Flex-generated lexer.

```
c
int lines = 1;
ofstream outlog;
ofstream outerr;
int error_count = 0;
```

- `lines` : global line counter, incremented by the lexer on every `\n` ; used to tag every log/error line with `At line no: N` .
- `outlog` / `outerr` : the two required output streams — `<id>_log.txt` (grammar trace + symbol table dumps) and `<id>_error.txt` (semantic errors only).
- `error_count` : running tally, printed at the end of both files.

c

```
symbol_table *symtbl = nullptr;
string activeType = "";
vector<pair<string,int>> varCollection;
vector<pair<string,string>> paramCollection;
string funcName = "";
string funcReturnType = "";
bool insertParamsToNextScope = false;
```

These are the **cross-rule state variables** — Bison actions can't easily pass complex accumulated data (like "all the variable names declared so far in this statement") purely through `$$`, so the grammar uses global scratch variables instead:

- `symtbl`: the single symbol table instance for the whole program (holds the current scope + parent chain).
- `activeType`: last-seen base type (`int` / `float` / etc.) — set in `type_specifier`, though in practice the grammar mostly re-passes `$1->getname()` explicitly rather than relying on this.
- `varCollection`: accumulates `(name, arraySize)` pairs while parsing a `declaration_list`, where `arraySize == -1` means "not an array". Flushed into the symbol table when the enclosing `variable_decl` reduces.
- `paramCollection`: accumulates `(type, name)` pairs while parsing a function's `param_list`. Used both to attach a parameter list to the function's `symbol_info` (for later call-site checking) and, via `insert_params_to_scope`, to insert the parameters as local variables in the function body's scope.
- `funcName` / `funcReturnType`: bookkeeping only (currently write-only — set but not read elsewhere; harmless leftover scaffolding).
- `insertParamsToNextScope`: a one-shot flag set the moment a function header is reduced, so the *next* `LCURL` (the function's opening brace) knows to dump `paramCollection` into the freshly-created scope. This solves the ordering problem: the parameter list is parsed and reduced *before* the scope for the function body even exists.

Helper functions

```
c
void log_error(const string &message, int line_no = -1)
{
    int report_line = (line_no >= 0) ? line_no : lines;
    outerr << "At line no: " << report_line << " " << message << endl << endl;
    outlog << "At line no: " << report_line << " " << message << endl << endl;
    error_count++;
}
```

Central error-reporting routine. Writes the **same message to both files**, defaults to the current global `lines` but allows an explicit override (used for mid-rule actions where `lines` may have already moved past the relevant source line — e.g. multiple-parameter-declaration errors). It never throws or exits, so parsing always continues — satisfying the "must not stop on first error" requirement.

```
c
void insert_params_to_scope(int scope_line)
{
    if (!insertParamsToNextScope) return;
    for (auto &pc : paramCollection)
    {
        if (pc.second.empty()) continue; // unnamed parameter, skip inserting
        symbol_info *ps = new symbol_info(pc.second, "ID");
        ps->set_data_type(pc.first);
        ps->set_symbol_kind("variable");
        if (!symtbl->insert(ps)) {
            log_error("Multiple declaration of variable " + pc.second + " in scope " +
                    scope_line);
            delete ps;
        }
    }
    insertParamsToNextScope = false;
}
```

Called from inside the `compound_statement` rule's mid-rule action right after `symtbl->enter_scope()`. It's guarded by the flag so ordinary (non-function) blocks don't accidentally re-insert stale parameters. Each named parameter becomes a normal `"variable"` symbol in the function's top scope; unnamed parameters (`void f(int)`) are skipped since they have no identifier to declare. Duplicate parameter names are caught the same way as duplicate variable declarations.

c

```
string infer_type(const string &left, const string &right)
{
    if (left == "error" || right == "error") return "error";
    if (left == "float" || right == "float") return "float";
    if (left == "int" || right == "int") return "int";
    return "void";
}
```

Implements C's usual arithmetic conversion for `+` / `-` (and reused nowhere else): error dominates, then float dominates over int, else int, with `void` as the fallback (which should never legitimately occur since void operands are checked separately beforehand).

c

```
bool is_zero_literal(const string &text)
{
    char *end = nullptr;
    double value = strtod(text.c_str(), &end);
    return end != text.c_str() && *end == '\0' && value == 0.0;
}
```

Static (compile-time, syntactic) check for "is this operand textually the literal zero?" — used only for the `/` case of `MULOP` to catch obvious `x / 0` patterns. `strtod` is used instead of a plain string match so `0`, `0.0`, `00.000` etc. are all recognized; the `*end == '\0'` check ensures the *whole* token was numeric (so a variable named e.g. `x0` isn't misread).

c

```
void yyerror(char *s) { log_error(s); }
```

Bison's standard syntax-error callback, routed through the same `log_error` so syntax errors also land in both output files and bump `error_count`.

2. Token & Precedence Declarations

c

```
%token IF ELSE FOR WHILE DO BREAK INT CHAR FLOAT DOUBLE VOID RETURN SWITCH CA  
      DEFAULT CONTINUE PRINTLN ADDOP MULOP INCOP DECOP RELOP ASSIGNOP LOGICC  
      LPAREN RPAREN LCURL RCURL LTHIRD RTHIRD COMMA SEMICOLON CONST_INT CONS
```

```
%nonassoc LOWER_THAN_ELSE
```

```
%nonassoc ELSE
```

- Declares every terminal produced by the lexer.
- The two `%nonassoc` lines resolve the classic **dangling-else ambiguity**: the `IF LPAREN expression RPAREN statement` rule is given the artificial low-precedence token `LOWER_THAN_ELSE` (via `%prec`), and `ELSE` is given higher precedence, so Bison always prefers to shift an `ELSE` and attach it to the nearest unmatched `if` rather than reducing early. Neither is truly associative — `%nonassoc` is used purely as a precedence trick here, not because `if / else` can be chained associatively.

3. Top-Level Rules: `start`, `program`, `unit`

```
start : program { print "Symbol Table" header; symtbl->print_all_scopes(outlog); }
```

The grammar's true start symbol. After the whole program has reduced, it dumps whatever is left in the symbol table (normally just the global scope, since every nested scope was already printed-and-popped by `compound_statement`).

```
program : program unit | unit
unit    : variable_decl | func_definition
```

Standard left-recursive "list of top-level declarations" pattern — a C source file is a sequence of global variable declarations and function definitions. Each level's action just concatenates the pretty-printed source text (`$1->getname()+...`) purely for the log file; no semantic checking happens here since it's already been done in the children.

4. `func_definition`

Two alternatives — with parameters and without:

```
func_definition : type_specifier ID LPAREN param_list RPAREN
  { /* mid-rule: register the function symbol BEFORE the body is parsed
  */ }
  compound_statement
  { /* final: build the pretty-printed text, reset per-function state */
  }
```

Mid-rule action (fires right after `RPAREN` is reduced, before `compound_statement` starts):

- `funcReturnType`, `funcName` recorded.
- `insertParamsToNextScope = true` — arms the flag consumed by `compound_statement`'s `LCURL` action.
- Builds a `symbol_info` for the function itself: `symbol_kind = "function"`, `data_type` = return type, and — crucially — `set_parameters(paramCollection)` attaches the full parameter type list so later call-sites can validate arity/types against it.
- `symtbl->insert(f)` inserts the function into the **current (enclosing/global) scope**. If it fails (name already used), logs `"Multiple declaration of function <name>"` and deletes the orphaned node.

Final action (after `compound_statement` reduces): purely cosmetic pretty-printing (`type name(params)\n{body}`) for the log file, then resets `paramCollection`, `funcName`, `funcReturnType` for the next function.

```
| type_specifier ID LPAREN RPAREN { ... } compound_statement { ... }
```

Same logic but for a **zero-parameter function**. Note `paramCollection.clear()` happens explicitly here (since no `param_list` rule ran to populate/clear it), guaranteeing a stale parameter list from a previous function definition can't leak in.

5. `param_list`

Four alternatives, all *left-recursive*, each pushing into `paramCollection` (note: this list is later consumed twice — once for `f->set_parameters()`, once for `insert_params_to_scope`):

```
param_list : param_list COMMA type_specifier ID      -- named parameter, 
            appended
            | param_list COMMA type_specifier
            appended (name = "")                    -- unnamed parameter,
            | type_specifier ID                      -- FIRST named
parameter: paramCollection.clear() then push
            | type_specifier                        -- FIRST unnamed
parameter: paramCollection.clear() then push
```

The `.clear()` calls on the two single-symbol (base-case) alternatives are what reset the global accumulator at the start of a new function's parameter list, preventing parameters from a previous function definition from bleeding into the next one. The two-symbol alternatives keep appending onto whatever the base case started.

6. `compound_statement` — Scope Management

```
compound_statement : LCURL
{
    int scope_line = lines-1;
    symtbl->enter_scope();
    outlog << "New ScopeTable with ID " << ... << " created";
    insert_params_to_scope(scope_line);
}
statements RCURL
{
    $$ = new symbol_info("{\n"+$3->getname()+"\n}", "comp_stmnt");
    symtbl->print_all_scopes(outlog);
    outlog << "Scopetable with ID " << ... << " removed";
    symtbl->exit_scope();
}
```

This is the **only place scopes are created/destroyed**. The mid-rule action after `LCURL` fires immediately when the opening brace is shifted+reduced (a Bison "mid-rule action"), *before* any statements inside the block are parsed — so `enter_scope()` happens first, then (if this `compound_statement` is a function body) the just-collected function parameters are inserted into that brand-new scope via `insert_params_to_scope`.

The final action (after `RCURL`) does three things in order: builds the pretty-printed block text, **dumps every currently-active scope table to the log** (so the log shows the block's local symbol table at the moment it's about to be discarded), announces removal, then actually calls `exit_scope()` to pop it. This ordering matters — printing must happen *before* `exit_scope()` destroys the data.

```
| LCURL { ... } RCURL { ... }
```

Identical logic for an **empty block** (`{ }` with no statements) — separate alternative because the grammar has no way to make `statements` itself nullable without introducing ambiguity, so an explicit empty-body rule is written out.

7. `variable_decl` and `type_specifier`

```
variable_decl : type_specifier declaration_list SEMICOLON
{
    if ($1->getname() == "void") log_error("variable type can not be
void ");
    for (auto &vc : varCollection) {
        symbol_info *vs = new symbol_info(vc.first, "ID");
        if (vc.second >= 0) { vs->set_symbol_kind("array"); vs-
>set_array_size(vc.second); }
        else { vs->set_symbol_kind("variable"); }
        vs->set_data_type($1->getname() == "void" ? "error" : $1-
>getname());
        if (!symtbl->insert(vs)) {
            log_error("Multiple declaration of variable " + vc.first);
            delete vs;
        }
    }
    varCollection.clear();
}
```

This is where declared variables actually get inserted into the symbol table (uniqueness checking's insertion side). `declaration_list` (below) has already populated `varCollection` with `(name, size)` pairs where `size == -1` marks a plain scalar and `size >= 0` marks an array of that length. For each entry:

- Build a fresh `symbol_info`, classify it as `"array"` (with size) or `"variable"`.
- **Void-variable guard:** `void x;` is illegal in C, so if the declared type itself is `"void"`, an error is logged once for the whole declaration, and every variable in the list is given the poisoned type `"error"` (rather than the literal string `"void"`) so it doesn't cause a second, misleading `"operation on void type"` error the first time it's *used* — a deliberate design choice to avoid duplicate/confusing error cascades.
- `symtbl->insert(vs)` performs the actual uniqueness check for this scope; failure → log + delete (no leak).
- `varCollection.clear()` resets the accumulator for the next `variable_decl`.

```
type_specifier : INT | FLOAT | VOID | CHAR
```

Four trivial terminal-to-nonterminal conversions. Each sets the scratch `activeType` global (largely vestigial, since callers use `$1->getname()` directly) and returns a `symbol_info` of kind `"type"` whose name is the type string (`"int"`, `"float"`, `"void"`, `"char"`) — this string is what every later `get_data_type()` / `getname()` comparison (`== "int"`, `== "float"`, etc.) is built on.

8. `declaration_list`

```
declaration_list : declaration_list COMMA ID --   
push scalar  
                | declaration_list COMMA ID LTHIRD CONST_INT RTHIRD --  
push array  
            | ID --  
FIRST scalar: clear() then push  
            | ID LTHIRD CONST_INT RTHIRD --  
FIRST array: clear() then push
```

Same left-recursive "list with a clearing base case" pattern as `param_list`, but pushing `(name, size)` pairs into `varCollection` instead of `(type, name)` into `paramCollection`. `stoi($5->getname()) / stoi($3->getname())` converts the literal array-size text token into an integer for `set_array_size`. Note the array size here **must** be a literal `CONST_INT` (not an arbitrary expression) — consistent with C's requirement that a declared array's size be a compile-time constant.

9. `statements` and `statement`

```
statements : statement | statements statement
```

Simple statement-list accumulator; purely structural, no semantics beyond string concatenation for the log.

```
statement : variable_decl
          | expression_statement
          | compound_statement
          | FOR LPAREN expression_statement expression_statement expression
            RPAREN statement
          | DO statement WHILE LPAREN expression RPAREN SEMICOLON
          | BREAK SEMICOLON
          | CONTINUE SEMICOLON
          | IF LPAREN expression RPAREN statement %prec LOWER_THAN_ELSE
          | IF LPAREN expression RPAREN statement ELSE statement
          | WHILE LPAREN expression RPAREN statement
          | PRINTLN LPAREN ID RPAREN SEMICOLON
          | RETURN expression SEMICOLON
```

Covers the full statement grammar of the C subset. Only two alternatives do semantic work beyond building the pretty-printed statement text:

- `IF ... %prec LOWER_THAN_ELSE`: forces this reduction to have artificially low precedence so the dangling `else` always attaches to the innermost `if` (see the precedence section above) — this line is the actual use of that declared precedence.
- `PRINTLN LPAREN ID RPAREN SEMICOLON`: looks up `ID` in the symbol table and logs `"Undeclared variable <name>"` if it isn't found — this is the one place besides `variable : ID` where an existence check happens directly against a raw `ID` token (since `println(x)` takes a bare identifier, not a full `variable / expression`, per the grammar).
- `FOR`, `DO/WHILE`, `WHILE`, `IF` don't type-check their condition expression against `"int"` explicitly here — their condition is just an `expression`, whose own sub-rules (`logic_expression`, `rel_expression`, etc.) have already resolved/flagged type errors (e.g. void-in-condition) internally; the `statement` rule itself just re-assembles the text

10. `expression_statement`

```
expression_statement : SEMICOLON  
                    | expression SEMICOLON
```

An empty statement (`;`) or an expression followed by `;` (e.g. `x = 5;`, `foo();`, `i++;`). The second alternative propagates the expression's `data_type` upward via `$$->set_data_type($1->get_data_type())`, though in practice nothing above `statement` reads that type further (it's mainly there for consistency/possible future use, e.g. checking a `RETURN`'s expression type against `funcReturnType`, which — note — the current grammar does not actually do: `RETURN expression SEMICOLON` builds the text but never compares `$2->get_data_type()` to the enclosing function's declared return type).

11. `variable` — the Core of Uniqueness & Array Checking

```
variable : ID
{
    $$ = new symbol_info($1->getname(), "varbl");
    symbol_info temp($1->getname(), "ID");
    symbol_info *existing = symtbl->lookup(&temp);
    if (existing == nullptr) {
        log_error("Undeclared variable " + $1->getname());
        $$->set_data_type("error");
    } else if (existing->get_symbol_kind() == "array") {
        log_error("variable is of array type : " + $1->getname());
        $$->set_data_type("error");
    } else {
        $$->set_data_type(existing->get_data_type());
    }
}
```

This is the existence check for scalar variable use. `symtbl->lookup()` walks from the current scope outward through enclosing scopes (implementing normal C scoping/shadowing). Three outcomes:

1. Not found anywhere → undeclared-variable error, type becomes `"error"`.
2. Found, but it's actually an *array* being used without an index (e.g. `x` where `int x[5];` was declared) → "array used as scalar" error, type `"error"`.
3. Found and is a genuine scalar → inherit its declared data type.

```

| ID LTHIRD expression RTHIRD
{
    ... lookup as above ...
    else if (existing->get_symbol_kind() != "array") {
        log_error("variable is not of array type : " + $1->getname());
        $$->set_data_type("error");
    } else {
        string indexType = $3->get_data_type();
        if (indexType == "error")    $$->set_data_type("error");
        else if (indexType != "int") { log_error("array index is not
of integer type : " + name); $$->set_data_type("error"); }
        else                        $$->set_data_type(existing-
>get_data_type());
    }
}
}

```

The array-access counterpart. First re-runs the same existence lookup; if found but **not** an array (i.e., someone wrote `x[3]` where `x` is a scalar), logs the mirror-image error. If it genuinely is an array, then checks the **index expression's type** — must resolve to `"int"` (integers only; `float` / `char`-typed indices are rejected). On success, the resulting `variable` node's type is the array's *element* type (i.e., indexing strips the "array"-ness, matching normal semantics where `arr[i]` is a scalar of the base type, not an array itself).

12. `expression` — Assignment

```
expression : logic_expression
{ $$->set_data_type($1->get_data_type()); }
| variable ASSIGNOP logic_expression
{
    string leftType = $1->get_data_type();
    string rightType = $3->get_data_type();
    if (leftType=="error" || rightType=="error")    $$->set_data_type("error");
    else if (leftType=="void" || rightType=="void") {
        log_error("operation on void type "); $$->set_data_type("error"); }
    else if (leftType=="int" && rightType=="float") {
        log_error("Warning: ... float value into ... integer type "); $$->set_data_type("int"); }
    // (leftType stays as the default $$ type otherwise - e.g.
    float=int, int=int, float=float all pass silently)
}
```

Top of the expression hierarchy. The plain `logic_expression` alternative is the pass-through case (an expression that isn't an assignment). The `ASSIGNOP` alternative implements Type Checking rule 1 (assignment consistency) and Type Conversion rule 1 (float→int warning):

- error propagates silently (no duplicate message),
- assigning to/from `void` is a hard error,
- `int = float` is legal but lossy → warning only, forced result type `"int"`,
- everything else (the implicit widening `float = int`, or matching types) is accepted with no message. The resulting node's type defaults to the *left-hand variable's* type, matching C's rule that an assignment expression's type/value is that of the (converted) left-hand side.

13. `logic_expression` — `&&` / `||`

```
logic_expression : rel_expression
                  | rel_expression LOGICOP rel_expression
{
    if error-in-either -> "error"
    else if void-in-either -> log "operation on void type", "error"
    else -> "int"
}
```

Implements **Type Conversion rule 2**: the result of a `LOGICOP` (`&&`, `||`) operation is always forced to `"int"` (C has no boolean type; logical results are ints), after the same error/void guard pattern used throughout the file.

14. `rel_expression` — Relational Operators

```
rel_expression : simple_expression
                | simple_expression RELOP simple_expression
{ same error/void/-> "int" pattern as logic_expression }
```

Implements the other half of Type Conversion rule 2: `RELOP` (`<`, `>`, `==`, etc.) results are also always `"int"`. Structurally identical logic to `logic_expression`, just one level down the precedence chain (this is how the grammar encodes that `&&` / `||` bind looser than `<` / `>` / `==`).

15. `simple_expression` — `+` / `-` (ADDOP)

```
simple_expression : term
                  | simple_expression ADDOP term
{
    if error-in-either -> "error"
    else if void-in-either -> log "operation on void type", "error"
    else -> infer_type(leftType, rightType) // error/float/int/void
per helper
}
```

Implements arithmetic type promotion for `+` / `-` : uses the `infer_type()` helper described in the preamble section to decide `int` vs `float` result (float "poisons" int, matching normal C arithmetic conversions).

16. `term` — `*` / `/` / `%` (MULOP)

```
term : unary_expression
      | term MULOP unary_expression
{
    if ($2->getname() == "%") {
        $$->set_data_type("int"); // modulus always int
    } else if ($2->getname() == "/") {
        error/void guard, THEN:
        if (is_zero_literal(right operand text)) { log "Division by 0
"; -> "error"; }
        else -> (float if either operand float, else int)
    } else { // "*"
        error/void guard, THEN -> (float if either operand float, else
int)
    }
}
```

This single rule handles **three different operators** because the lexer collapses `*`, `/`, `%` into one token class `MULOP`, distinguished at semantic-action time by inspecting the operator's literal text (`$2->getname()`). Implements:

- **Type Checking rule 3** (modulus operands): the code currently just forces the result to `int` unconditionally rather than first verifying both operands actually *are* integers — this is the gap flagged earlier (a fully spec-compliant version would check `leftType=="int" && rightType=="int"` first and emit an error like `"non-integer operand of modulus"` otherwise).
- **Type Checking rule 4** (division/modulus by zero): only implemented for `/`, via the static `is_zero_literal()` text check against the *right*-hand operand's printed representation — catches literal-zero divisors (`x / 0`) but not runtime-computed zero divisors.
- Ordinary `*` and (non-zero) `/` both apply the same float-dominates-int promotion as `ADDOP`.

17. `unary_expression` — Unary `+`/`-`, `!`, and pass-through

```
unary_expression : ADDOP unary_expression
  { error->error; void->log+error; else-> same type as operand }
| NOT unary_expression
  { error->error; void->log+error; else-> forced to "int" }
| factor_info
  { pass-through: same type as $1 }
```

- Unary `+`/`-` preserves the operand's type (an `int` stays `int`, a `float` stays `float`) once error/void are handled.
- `NOT` (logical negation `!`) always yields `"int"`, consistent with the "RELOP/LOGICOP result is int" rule elsewhere.
- `factor_info` is a trivial pass-through wrapper node (exists purely so `factor` output has one extra layer to print in the grammar trace, matching the reference grammar's non-terminal naming).

18. `factor` — Atoms: Variables, Calls, Literals, `++` / `--`

```
factor : variable
{ pass-through of variable's type }
```

Simple reuse of everything already validated in the `variable` rule.

```
| ID LPAREN argument_list RPAREN
{
    lookup ID;
    if not found          -> log "Undeclared function: <name>", type
= "error"
    else if kind != "function" -> log "<name> is not a function", type
= "error"
    else {
        type = function's declared return type;
        params = function symbol's stored parameter list
        arguments = $3's collected argument types (from argument_list -
> arguments)
        if (params.size() != arguments.size())
            log "Inconsistencies in number of arguments in function
call: <name>"
        else
            for each position i:
                if arguments[i].type != "error" && arguments[i].type !=
params[i].type
                    log "argument <i+1> type mismatch in function call:
<name>"
    }
}
```

This is the **function-call semantic check**, covering:

- **Function-Parameter rule 2** (non-function identifiers can't be called) and the undeclared-function case.
- **Type Checking rule 5** (arguments consistent with the definition) and **Function-Parameter rule 1** (correct count and types): compares sizes first (arity check) — only if they match does it walk position-by-position comparing declared parameter types to actual argument types, skipping the comparison for any argument that's already `"error"` (again, to avoid cascading duplicate messages from an already-broken sub-expression).
- Note the call's own resulting type is always set to the function's return type regardless of whether an arity/type mismatch was found — the error is *logged* but doesn't further poison the call's type to `"error"`, meaning a badly-called function still contributes its declared return type to any enclosing expression (a minor design choice; could arguably be `"error"` instead, but doesn't affect correctness since the error was already reported).

```
| LPAREN expression RPAREN
  { pass-through type; adds parentheses to the pretty-printed text only }
| CONST_INT   { type = "int" }
| CONST_FLOAT { type = "float" }
| variable INCOP { type = variable's type } // e.g. x++
| variable DECOP { type = variable's type } // e.g. x--
```

Literals get their type directly from which token matched (`CONST_INT` → `int`, `CONST_FLOAT` → `float`). Post-increment/decrement simply preserve the underlying variable's type (no error/void checking is applied here — e.g. `voidVar++` wouldn't currently be separately flagged at this rule, though `voidVar` itself would already have been marked `"error"` by the `variable` rule if it were genuinely void, since `void`-typed variables are disallowed at declaration time and get type `"error"` instead).

19. `argument_list` and `arguments` — Building the Call's Argument-Type Vector

```
argument_list : arguments      { $$->set_parameters($1->get_parameters()); }
                | (empty)      { $$ = new symbol_info("", "arg_list"); }
// zero-argument call: foo()
```

Wraps `arguments` (below) and also handles the zero-argument case directly (an empty production), so `foo()` still produces a valid `argument_list` node whose parameter vector is simply empty (matched against a zero-length `params` in the `factor` rule's arity check).

```
arguments : arguments COMMA logic_expression
{
    argumentType = $3's type;
    if (argumentType == "void") { log "operation on void type ";
argumentType = "error"; }
    append (argumentType, "") to the accumulated list carried on $1's
parameters;
}
| logic_expression
{
    argumentType = $1's type;
    if (argumentType == "void") { log "operation on void type ";
argumentType = "error"; }
    start a new list with just (argumentType, "");
}
```

Builds up the ordered vector of argument types (names are irrelevant for arguments, hence the empty second element of each pair — reusing the same `pair<string,string>` type as `paramCollection` for convenience). This is also where "a void function cannot be called as part of an expression" is enforced *for the argument position specifically*: if you write `foo(bar())` where `bar` is void, `bar()`'s resulting type is `"void"`, and *this* rule catches it right here, converting it to `"error"` before it's compared against the callee's expected parameter type (so it correctly produces one clean error rather than a

20. %% and main()

```
c
int main(int argc, char *argv[])
{
    if (argc != 2) { ... }
    yyin = fopen(argv[1], "r");
    outlog.open("22201295_log.txt", ios::trunc);
    outerr.open("22201295_error.txt", ios::trunc);
    if (yyin == NULL) { ... }

    symtbl = new symbol_table(10);
    outlog << "New ScopeTable with ID " << symtbl->get_current_scope_id() <<

    yyparse();

    outlog << endl << "Total lines: " << lines << endl;
    outlog << "Total errors: " << error_count << endl;
    outerr << "Total errors: " << error_count << endl;

    outlog.close(); outerr.close(); fclose(yyin);
    delete symtbl;
    return 0;
}
```


Summary of Where Each Spec Requirement Is Implemented

Spec Requirement	Rule(s)
Assignment operand consistency	<code>expression : variable ASSIGNOP</code> <code>logic_expression</code>
Array index must be integer	<code>variable : ID LTHIRD expression RTHIRD</code>
Modulus operands must be integers	<code>term : term MULOP unary_expression (%</code> branch — currently incomplete, see caveat)
Division/modulus by zero	<code>term : term MULOP unary_expression (/</code> branch, via <code>is_zero_literal</code>)
Function call argument consistency	<code>factor : ID LPAREN argument_list RPAREN</code>
Void function not usable in expression	<code>arguments</code> rules (void→error conversion) + void-guards in every operator rule
Float assigned to int → warning	<code>expression : variable ASSIGNOP</code> <code>logic_expression</code>
RELOP/LOGICOP result is int	<code>rel_expression</code> , <code>logic_expression</code>
Undeclared variable use	<code>variable : ID</code> , <code>variable : ID LTHIRD ...</code> <code>RTHIRD</code> , <code>PRINTLN</code> statement
Duplicate declaration in same scope	<code>syntbl->insert()</code> failure paths in <code>variable_decl</code> , <code>func_definition</code> , <code>insert_params_to_scope</code>
Index used with non-array / array used without index	<code>variable : ID</code> and <code>variable : ID LTHIRD</code> <code>expression RTHIRD</code>
Function call argument count & type	<code>factor : ID LPAREN argument_list RPAREN</code>
Call on non-function identifier	Same rule, <code>symbol_kind != "function"</code> check

Lab 4

Good, I have enough grounding in your actual implementation. Let me build a solid, code-specific viva sheet. Now I have a good picture of your implementation. Here's a 50-question viva bank built around **your actual code** (`ast.h`, `22299522.y`, `three_addr_code.h`), grouped by theme so it flows like a real viva.

A. Fundamentals / Conceptual (Q1–12)

1. What is intermediate code, and why does a compiler use it instead of going straight from source to machine code? It's a representation between the source and target languages — machine-independent, simpler than source, closer to machine code. It lets you retarget the backend for different machines and apply machine-independent optimizations without touching the front end.

2. What is three-address code (TAC)? Why "three-address"? A linear IR where each instruction has at most three operands/addresses (e.g., `t0 = a + b` — two operands + one result). It's a compromise between high-level expressions and single-operand machine instructions.

3. What is an AST, and how does it differ from a parse tree? An Abstract Syntax Tree keeps only semantically significant nodes (operators, operands, control structures) and discards grammar artifacts like parentheses and intermediate non-terminals that a full parse/derivation tree would retain. It's more compact and closer to the program's logical structure.

4. Why build an AST first instead of generating TAC directly during parsing? Building the AST decouples syntax recognition from code generation — you get a structure you can traverse (possibly multiple times), do semantic checks or optimizations on, and generate code from in a controlled post-order manner, rather than being locked into whatever order Yacc reduces rules in.

5. In your design, is TAC generated during parsing or after the AST is fully built? After. The parser (`.y`) only *builds* the AST via `set_ast_node/get_ast_node` on each production; actual code generation happens afterward when `main()` calls `ThreeAddrCodeGenerator::generate()` on the completed `ast_root`.

6. What design pattern does your `generate_code()` scheme resemble? It's essentially the Visitor/Interpreter pattern implemented via virtual dispatch — every node overrides a common

virtual method, and traversal recursively calls the same method on children, so the "visit" logic lives inside each node class rather than a separate visitor.

7. Why is `generate_code` declared `virtual` and `pure (= 0)` in `ASTNode`? To force every concrete node subclass to provide its own code-generation logic, and to allow polymorphic calls — a `BlockNode` holding `StmtNode*` pointers can call `generate_code` without knowing the concrete runtime type (`IfNode`, `WhileNode`, etc.).

8. Why does every `generate_code` return a `string`? It returns the name of the temporary (or variable) holding the result of that subexpression, so the parent node can use it as an operand — this is how results are threaded up the tree during a post-order traversal.

9. What are the three output files your compiler produces, and what does each contain? `log.txt` — matched grammar rules and source segments, plus AST/parse info, with a line & error count at the end. `error.txt` — error messages, with error count at the end. `code.txt` — the generated three-address code.

10. What is `symbol_to_temp` used for? A `map<string, string>` that caches, for each variable name, the temporary that currently holds its up-to-date value — so repeated reads of the same variable in one flow don't necessarily re-emit a load, and assignments update the cached temp.

11. Why do `temp_count` and `label_count` need to be passed by reference (`int&`) through every `generate_code` call? Because temporaries and labels must be uniquely numbered across the *entire* program, not per-node. Passing by reference lets every recursive call share and increment the same global counters instead of restarting from `t0/L0` in each subtree.

12. What would go wrong if `temp_count` were passed by value instead of by reference? Every node would see its own local copy starting from whatever value it received, so nested/sibling calls would reuse the same temporary names (e.g., `t0` used in two different subexpressions), producing incorrect, colliding TAC.

B. AST Node Design (Q13–24)

13. Walk me through the class hierarchy in `ast.h`. `ASTNode` is the abstract base. `ExprNode` and `StmtNode` both inherit from it (also abstract, with a shared `generate_code` signature). Expressions: `VarNode`, `ConstNode`, `BinaryOpNode`, `UnaryOpNode`, `AssignNode`, `FuncCallNode` inherit `ExprNode`. Statements: `ExprStmtNode`, `BlockNode`, `IfNode`, `WhileNode`, `ForNode`, `ReturnNode`, `DeclNode` inherit `StmtNode`. `FuncDeclNode`

and `ProgramNode` inherit `ASTNode` directly (they aren't expressions or statements). `ArgumentsNode` is a helper, also `ASTNode`.

14. Why does `ExprNode` store a `node_type` string field, while `StmtNode` doesn't?

Expressions have a value and hence a type (int, float, void...) needed for type-aware code generation/semantic checks (e.g., `FuncCallNode` checks `node_type == "void"`). Statements (like `if`, `while`) don't produce a value, so there's no type to track.

15. Explain how `VarNode` handles array vs. scalar variables with a single class. It stores an optional `ExprNode* index`, defaulted to `nullptr`. `has_index()` tells the difference. In `generate_code`, if `index` is set, it recursively generates code for the index expression and emits `tX = name[index_temp]`; otherwise it treats it as a plain scalar lookup.

16. In `VarNode::generate_code`, for a plain scalar variable, walk me through the logic line by line. It always allocates a temp name string first, then checks `symbol_to_temp` for an existing cached temp for that name. If found, it returns that cached temp directly (the freshly allocated `temp` string is discarded — the counter was already incremented, so that temp number is "burned"/skipped). If not found, it emits `temp = name`, caches it, and returns it.

17. Isn't there a subtle inefficiency/bug in Q16 — the temp counter increments even when the cached path is taken? Yes — `temp_count++` happens unconditionally at the top before the cache check, so on a cache hit a temporary number is consumed but never printed/used, creating gaps in temp numbering (e.g., you might jump from `t2` to `t4`). It's not incorrect semantically, just leaves unused temp names.

18. What does `ConstNode::generate_code` do, and why might always emitting `tX = value` for a constant be considered slightly wasteful? It allocates a new temp and emits `tX = <literal>`. It's arguably wasteful because a constant could often be used directly as an operand without a temp (e.g., `t0 = 5 + b` instead of `t0=5; t1=t0+b`), but the trade-off here is uniformity — every `ExprNode` returns a temp, simplifying the parent's code.

19. Explain `BinaryOpNode::generate_code`'s evaluation order. It recursively generates code for `left` first, then `right`, capturing their result temporaries, then emits one instruction `result = left_temp op right_temp` using a new temp. This enforces left-to-right evaluation order and models the classic post-order traversal for expression trees.

20. How does `AssignNode::generate_code` differ for array LHS vs simple variable LHS? It always generates RHS code first. If `lhs->has_index()`, it generates the index code and emits `name[index_temp] = rhs_temp` (no update to `symbol_to_temp`, since arrays aren't cached). Otherwise it emits `name = rhs_temp` and, if `name` already has a cached temp, updates the map entry to point to `rhs_temp` so subsequent reads reflect the new value.

21. Why doesn't array assignment update `symbol_to_temp`? Because `symbol_to_temp` caches whole-variable values by name; array elements are indexed, so a single cached temp per array name can't represent "the value at index i" — caching would give incorrect stale values for other indices.

22. What does `AssignNode::generate_code` return, and why does that make sense for expressions like `x = y = 5`? It returns `rhs_temp`, i.e., the value assigned — mirroring how assignment is an expression with a value in C. This lets a chained/nested assignment use the result of an inner assignment as an operand elsewhere.

23. How does `DeclNode` differ fundamentally from the other statement nodes in terms of code generation? It doesn't generate any executable TAC — it only emits a `// Declaration: type name[size]` comment for documentation. Declarations affect the symbol table (handled elsewhere, during parsing) but produce no runtime instructions in TAC.

24. Why is `FuncDeclNode` not a subclass of `StmtNode` or `ExprNode`, but directly of `ASTNode`? A function declaration/definition isn't an expression (has no value) and isn't a statement inside a body (it's a top-level program unit), so it sits directly under the general `ASTNode` base alongside `ProgramNode`.

C. Control Flow Code Generation (Q25–34)

25. Walk me through `IfNode::generate_code` and the labels it creates. It allocates three labels: `true_label`, `false_label`, `end_label`. It generates the condition code, then emits `if cond_temp goto true_label` followed unconditionally by `goto false_label`. Under `true_label`: it emits the then-block, then `goto end_label`. Under `false_label`: it emits the else-block (if any). Finally emits `end_label`.

26. Why does `IfNode` need three labels instead of the two you'd expect (true/false)? Because after executing the then-branch, control must jump past the else-branch to a common `end_label`, otherwise execution would fall through into the else-block after finishing the then-block. The three labels are: where to go if true, where to go if false, and where both paths converge afterward.

27. Is `if cond goto true_label; goto false_label` code pattern efficient? Could it be simplified? It's a valid but slightly verbose "jump-on-true, else fall through" pattern doubled into explicit gotos; a more compact classical approach uses `ifFalse cond goto false_label` directly (one instruction), skipping the extra unconditional goto. Your version trades a couple of extra instructions for symmetry/clarity, which is acceptable for this course's purposes.

28. Walk me through `WhileNode::generate_code`. `start_label`: re-evaluates the condition each iteration; if true, jumps to `body_label` to execute the body then jumps back to `start_label`; if false, jumps to `end_label`. This is the classic "test-at-top" loop translation.

29. How does `ForNode::generate_code` combine the pieces of a C `for(init; cond; update)` loop? It generates `init` code once (outside the loop, unconditionally). Then it mirrors the while-loop pattern for the condition/body, and additionally generates `update` code at the end of each iteration body, right before jumping back to `start_label`.

30. In `ForNode`, what happens if `condition` is null (e.g., `for(;;)`)? `cond_temp` stays empty, so the code takes the `else` branch: it just emits `goto body_label` unconditionally — effectively an infinite loop with no condition check, correctly modeling `for(;;)`.

31. Why is `init` generated before the `start_label`, but `update` generated inside the loop body (after `body`)? `init` (e.g., `i = 0`) should run exactly once before the loop starts, so it's outside the labeled loop region. `update` (e.g., `i++`) must run once per iteration after the body but before re-testing the condition, so it belongs inside the loop, right before the jump back to `start_label`.

32. How do `IfNode`, `WhileNode`, and `ForNode` all avoid label collisions with each other, even in nested loops? They all share and increment the same `label_count` reference passed down from the root call, so every `LX` generated anywhere in the program (regardless of nesting depth) is globally unique.

33. What TAC does `ReturnNode` generate for `return`; versus `return expr`; For a bare `return`;, `expr` is null, so it emits just `return`. For `return expr`;, it generates the expression's code and emits `return expr_temp`, returning that temp as well.

34. Trace the exact TAC your generator would emit for `if (a > b) x = 1; else x = 0`; (assume it's the first code in the function). Something like:

```
t0 = a
t1 = b
t2 = t0 > t1
if t2 goto L0
goto L1
L0:
t3 = 1
x = t3
goto L2
L1:
t4 = 0
```

x = t4
L2:

D. Function Calls & Program Structure (Q35–41)

35. How does `FuncCallNode::generate_code` differ between void and non-void functions? It always first generates each argument's code and emits a `param arg_temp` instruction per argument. Then, if the function's `node_type` is "void", it just emits `call func_name, argc` with no result (returns empty string). Otherwise it allocates a new temp and emits `tX = call func_name, argc`, returning that temp.

36. Why emit `param` instructions before `call`, rather than passing all arguments directly in the call instruction? This mirrors the classic textbook three-address code convention for function calls (params pushed individually before the call), matching how many real compilers/back-ends handle parameter passing (e.g., via a stack or parameter list), and keeps each instruction to at most three addresses.

37. What role does `ArgumentsNode` play, and why does its destructor deliberately NOT delete its argument pointers? It's a temporary helper used only during parsing to accumulate a comma-separated argument list (`ExprNode*` pointers) before they get transferred into a `FuncCallNode`. Its destructor avoids deleting the arguments because ownership is transferred to `FuncCallNode`, which will delete them; double-deleting would cause a crash/undefined behavior.

38. Why does `FuncDeclNode::generate_code` call `symbol_to_temp.clear()` at the start? Because temp-to-variable caching is scoped per function — a variable named `x` in one function is unrelated to `x` in another, and stale cached temps from a previous function's local variables shouldn't leak into the next function's code generation.

39. Does clearing `symbol_to_temp` per function reset `temp_count` or `label_count` too? Should it? No — only the map is cleared; `temp_count/label_count` keep incrementing globally across the whole program. That's intentional here since temp/label names just need to be unique in the output file, not reset per function (unlike some implementations that restart numbering per function for readability).

40. What is `ProgramNode`'s role, and how does it relate to `ThreeAddrCodeGenerator::generate()`? `ProgramNode` is the AST root, holding a vector of top-level `ASTNode*` units (function declarations, global declarations). `ThreeAddrCodeGenerator::generate()` writes a header comment block to `code.txt`,

then calls `ast_root->generate_code(...)`, which iterates over all units and generates code for each — writing a trailer comment at the end.

41. In your grammar, how does `ProgramNode` actually get built across multiple `unit` reductions (e.g., multiple functions in a file)? Looking at your `.y`, the `program/unit_list` rules incrementally build a `ProgramNode*`: the first unit creates a new `ProgramNode` and adds itself via `add_unit`; each subsequent reduction reuses the existing `ProgramNode` (retrieved via `get_ast_node`) and appends the new unit, finally `ast_root` is assigned from the top-level reduction.

E. Yacc/Bison Integration (Q42–47)

42. What is `symbol_info*`, and why is `#define YYSTYPE symbol_info*` significant? It redefines Bison's semantic value type so every grammar symbol (`$1`, `$2`, `$$...`) is a pointer to your custom `symbol_info` class instead of a plain union — letting each token/non-terminal carry richer data (name, type, and critically, a pointer to its AST node) through `get_ast_node()/set_ast_node()`.

43. Explain the relationship between `get_ast_node()` / `set_ast_node()` on `symbol_info` and the AST classes. `symbol_info` acts as a wrapper/carrier during parsing — each grammar rule constructs the appropriate `ASTNode*` subclass, stores it into `$$'s symbol_info` via `set_ast_node()`, and parent rules retrieve children's AST nodes via `get_ast_node()` (often with a cast, e.g., `(ExprNode*)$1->get_ast_node()`) to build up their own node.

44. Give an example from your grammar of a "pass-through" rule versus a "node-creating" rule. Pass-through, e.g. `expression : logic_expression { $$->set_ast_node($1->get_ast_node()); }` — just forwards the child's AST node unchanged. Node-creating, e.g. the assignment rule, which builds a new `AssignNode` from the LHS/RHS AST nodes and casts appropriately.

45. Why are explicit casts like `(ExprNode*)$1->get_ast_node()` needed everywhere? Because `get_ast_node()` returns a generic `ASTNode*` (or similarly base-typed pointer) for uniformity across all grammar symbols, but the constructors of specific node types (e.g., `BinaryOpNode`) expect the more derived `ExprNode*/StmtNode*/VarNode*` type, so a downcast is required — the parser trusts the grammar to guarantee the actual dynamic type is correct.

46. Your `++/--` (increment/decrement) rules desugar into what AST structure? Looking at lines ~1239–1259: `i++` builds a `VarNode`, a `ConstNode("1", "int")`, wraps them in a `BinaryOpNode("+", varNode, oneNode, type)`, then wraps that in an `AssignNode(varNode, addNode, type)` — i.e., `i++` is literally rewritten as `i = i + 1` in the AST (and `i--` as `i = i - 1`), so no special TAC logic is needed for increment/decrement.

47. Where in the `.y` file does the actual `ThreeAddrCodeGenerator` get invoked, and why not inside the grammar actions? It's invoked in `main()` (around line 1361) after `yyparse()` finishes — `ThreeAddrCodeGenerator tacGen(ast_root, outcode); tacGen.generate();`. It's kept out of the grammar actions because code generation needs the *complete* AST (e.g., a while-loop's body might not be finished being parsed when its `WhileNode` is first created), so it must happen only after parsing is entirely done.

F. Memory Management, Edge Cases & Improvements (Q48–50)

48. How is memory managed for AST nodes, and where could this go wrong? Each composite node (`BinaryOpNode`, `IfNode`, `BlockNode`, etc.) owns its children via raw pointers and deletes them in its destructor, so destroying `ProgramNode` cascades and frees the whole tree. The risk: if a node is referenced from two places (e.g., a bug in grammar actions attaching the same child pointer to two parents), you'd get a double-free; also, on a parse *error* mid-construction, partially built nodes may leak since there's no exception-safe cleanup.

49. Your logical operators (`&&`, `||`) are implemented via `BinaryOpNode` just like arithmetic ones — what real-world optimization is this missing, and why does it matter? It's missing **short-circuit evaluation**: for `a && b`, real compilers avoid evaluating `b` if `a` is already false (and similarly skip `a`'s second operand check for `||`). Your current implementation always evaluates both operands unconditionally, which is simpler but incorrect/wasteful for expressions with side effects (e.g., `a && (b = compute())` would always compute `b` even when `a` is false).

50. If asked to extend this project to support optimization (e.g., constant folding), where would you add it, and why is having an AST first (rather than generating TAC directly) an advantage here? You'd add it either as a pre-pass that folds `ConstNode op ConstNode` combinations into a single `ConstNode` before code generation, or by adding a check inside `BinaryOpNode::generate_code` to detect if both `left/right` are `ConstNodes` and directly compute+emit the folded constant instead of two loads + an add. Having the AST as an explicit intermediate structure is what makes this possible — you can inspect/transform the tree in a

separate pass before touching `outcode`, which wouldn't be feasible if TAC were emitted directly during parsing with no structure to look back at.